

SOFTWARE DEVELOPMENT LIFE CYCLE AND APPLICATION

Concert Ticket Tracking System

Course: Software Design Laboratory

Group: 11 **Section:** CPE106L-4_B3

Members: Endriga, Christopher Mikel; Marquez, Sebastian; Ponce, Lawrence Jacob; Somera, John Dylan

1. Scenario

A local concert organization needs a simple way to manage customer ticket requests. The manual process can cause delays, difficulty locating requests, and inconsistent handling order. The proposed Concert Ticket Tracking System automates the queueing and tracking of ticket requests. Requests receive a queue number and are processed using First-In, First-Out (FIFO), so the earliest waiting request is served first.

2. System Proposal

The system is a small-scale desktop application built with Python and Tkinter, with SQLite as its database. It uses a 3-tier architecture: Presentation Layer, Business Logic Layer, and Data Access Layer. The system allows staff to add ticket requests, validate input, search records, view queue status, and process the next waiting request according to FIFO.

3. Requirements Gathering

Functional Requirements

ID	Requirement
FR-01	Add a customer's concert ticket request to the queue.
FR-02	Automatically assign a unique queue number.
FR-03	Validate required fields and ticket quantity before saving.
FR-04	Search ticket requests using queue number, customer, contact, event, or ticket type.
FR-05	Filter/search by Waiting or Served status.
FR-06	Process the next waiting request using FIFO.
FR-07	Store and retrieve ticket request data using SQLite.
FR-08	Display queue records in a user-friendly graphical interface.

Non-Functional Requirements

ID	Requirement
NFR-01	Usability: the interface should be simple and easy for staff to understand.
NFR-02	Reliability: valid requests should be stored persistently in SQLite.
NFR-03	Maintainability: code should be separated into presentation, business, and data tiers.
NFR-04	Data integrity: invalid required values must not be inserted into the database.
NFR-05	Performance: the small-scale application should respond promptly to normal queue operations.

4. Development — Minimum Features

Search Function: Searches by queue number, customer name, contact number, event name, or ticket type, with an optional status filter.

Data Validation: Checks required fields, customer-name length, contact-number format, valid ticket type, and quantity from 1 to 10.

User-Friendly Interface: Tkinter GUI provides a clear input form, queue controls, search controls, status count, and a table of ticket requests.

FIFO: The system processes the Waiting request with the smallest queue number first.

5. 3-Tier Design

Tier	Component	Responsibility
Presentation	presentation/main.py	Tkinter GUI, user input, messages, table display.
Business Logic	business/ticket_service.py	Validation, ticket rules, FIFO service coordination.
Data Access	data/ticket_repository.py + database.py	SQLite connection and CRUD/search operations.
Model	models/ticket.py	TicketRequest data structure.

6. Database Design

SQLite database: **concert_ticket_queue.db**. Main table: **ticket_queue**. Fields: id, queue_number, customer_name, contact_number, event_name, ticket_type, quantity, status, created_at, processed_at.

The queue_number is unique. Waiting and Served are the supported status values used by the application.

7. Use Cases (Minimum: 3)

Use Case	Actor	Main Flow	Expected Result
UC-01 Add Ticket Request	Staff	Enter customer, contact, event, ticket type, and quantity; click Add Ticket.	Data is validated, saved to SQLite, and assigned a queue number.
UC-02 Search Ticket	Staff	Enter a keyword and/or status; click Search.	Matching ticket records are displayed.
UC-03 Process Next Ticket	Staff	Click Process Next (FIFO).	The earliest Waiting request is marked Served and removed from the queue.

8. Testing Plan

Important exam requirement: Notify the instructor when it is time to perform testing. The formal test execution should be performed after this notification/approval.

Test	Input/Action	Expected Result
T01 Valid Add	Enter all valid fields; quantity 2.	Request is added and receives the next queue number.
T02 Empty Name	Leave customer name blank.	Validation error; record is not saved.
T03 Invalid Quantity	Enter quantity 0 or above 10.	Validation error; record is not saved.
T04 Search	Search by customer/event/queue number.	Matching records are shown.
T05 FIFO	Add several Waiting requests; process next repeatedly.	Queue numbers are served in ascending order.
T06 Empty Queue	Process when no Waiting request exists.	Informational message states the queue is empty.
T07 Persistence	Close and reopen the application.	Previously saved records remain in SQLite.

9. Challenges and Limitations

Challenges include maintaining FIFO behavior while storing data persistently, validating user input, and keeping the project simple enough for a small-scale practical exam. Limitations include a single local SQLite database, no online synchronization, no payment processing, no real ticket inventory integration, and no multi-user/network deployment.

10. SDLC Application

Planning: identify the manual queue problem and project scope. **Requirements Analysis:** define functional and non-functional requirements. **Design:** use 3-tier architecture, FIFO queue rules, GUI, and SQLite. **Development:** implement the Python application. **Testing:** execute the prepared test cases after notifying the instructor. **Deployment/Submission:** submit documentation, source code, database, and required group selfie.

11. Submission Checklist

Requirement	Status
Project Documentation (PDF)	Prepared

Source Code	Prepared
SQLite Database	Prepared
Search Function	Implemented
Data Validation	Implemented
User-Friendly Interface	Implemented
FIFO Queue Processing	Implemented
Minimum 3 Use Cases	Documented
Testing Plan	Prepared — instructor notification still required before
Challenges and Limitations	Documented
Group Selfie	Must be taken/submitted by the group

Rubric alignment: The documentation is organized and complete; the code uses a 3-tier structure with clear indentation and documentation; findings/comments are tied to test data and observed results; and the report uses formal, readable grammar and sentence construction.